实验六：Verilog 编程：咏叹调（变音器）

一、实验目的

用 Verilog 语言设计变音器（实现音量高低随时间变化）。

二、 实验内容

设计一个声音高低随时间变化的变音器。

1）需实现的效果

从无声到响亮，再从响亮到无声。

2）原理

蜂鸣器的声音强度与控制电流成正比。在特定频率下，占空比如果为 0，则蜂鸣器静音；如果为 100%，则蜂鸣器满档，最响亮。如果使得占空比从 0 逐步变化到 100%，再从 100%逐步变化到 0，就可以实现蜂鸣器的咏叹效果。

3）设计要求

a）假设时钟频率为 5KHz，即周期为 0.2 毫秒

b）音量提高的过程耗时 2 秒，最强音处维持 1 秒，音量降低的过程耗时 2 秒，静音处维持 1 秒。周而复始。

c）声音强度可分为 11 档，（0，10%，20%，…, 90%, 100%），因此 2 秒时间分为 10 份，即每档占时 0.2 秒。

d）PWM（脉宽调制）频率自定。

三、实验步骤

1. 在 Vivado 开发环境中新建工程时，首先需要点击主界面中的"Create Project"按钮启动创建向导，随后根据系统提示逐步完成项目配置。

关键步骤包括：为项目命名（建议使用具有描述性的名称）并指定存储目录（推荐选择空间充足的硬盘路径），这些基础设置将直接影响后续设计文件的管理效率，如图 1 所示。

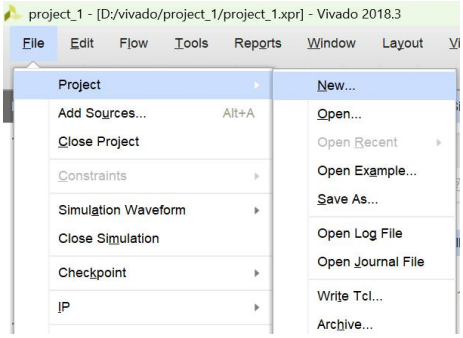


图 1 新建工程

2.在项目创建向导中，需要选择工程类型，如图 2 所示。

建议默认勾选"RTL Project"选项（Register Transfer Level 项目），这是最常见的数字电路设计类型，适用于从基础组合逻辑到复杂时序系统的各类硬件开发需求。该选项允许用户后续添加设计源文件并进行综合与实现，是绝大多数 Verilog/VHDL 设计项目的标准选择方案。

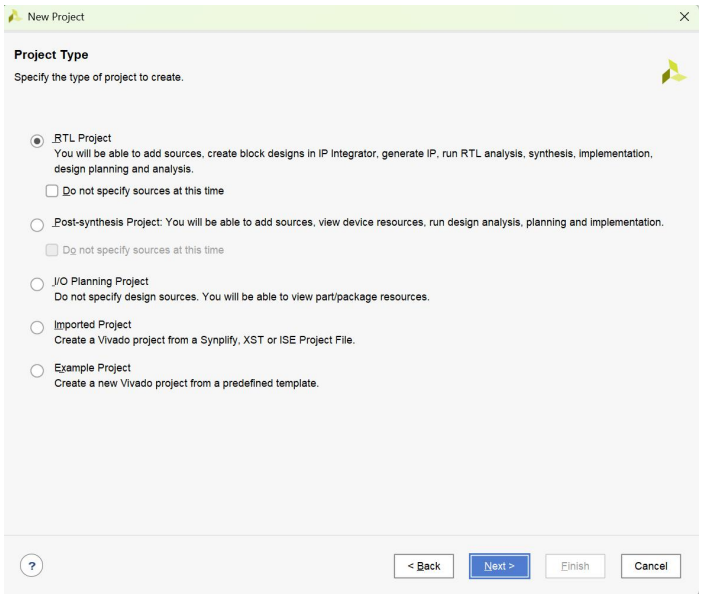


图 2 项目创建向导

3.在项目配置过程中，需要完成两个关键步骤：

首先处理设计源文件，用户可立即导入现有的 Verilog/VHDL 文件（支持.v/.vhd 等格式），若暂未准备源文件，系统允许在项目创建后通过"Add Sources"功能随时补充；

其次需指定目标器件，根据实际使用的开发板型号（如 Xilinx Artix-7 系列或 Zynq-7000 系列），在器件选择界面通过筛选封装、速度等级等参数精确匹配对应的 FPGA 芯片型号，确保硬件平台与设计项目的兼容性，如图 3 所示。

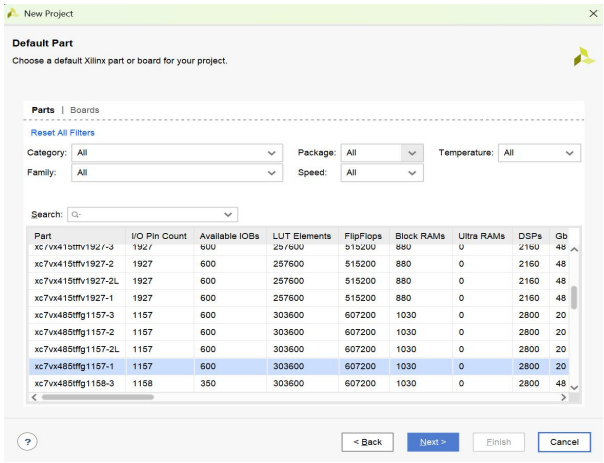


图 3 新工程默认部分

4.点击 finish，如图 4 所示。

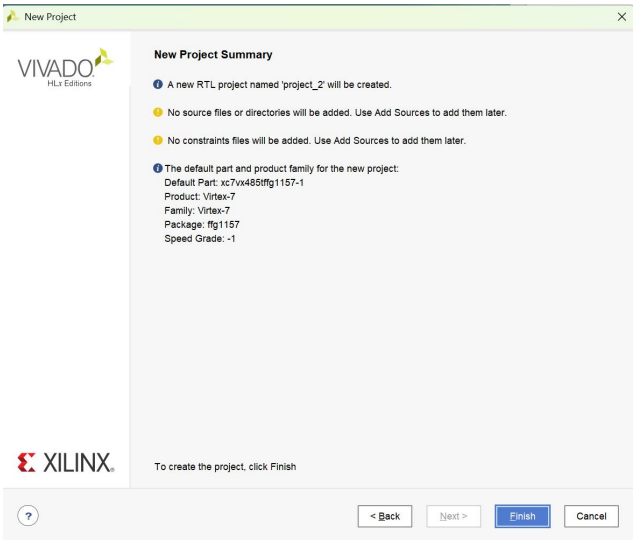


图 4 创建完成

5.Vivado 工程配置阶段，当需要添加设计文件时，用户应点击"Add Sources"界面中的"+"按钮，在弹出的选项菜单中选择"Create Design Sources"来新建设计文件。此时系统会进入文件创建向导，选择"Create File"选项后，可以指定文件名（如 module_name.v）、文件类型（Verilog 或 VHDL）以及存储路径。这个流程允许设计者在项目初始阶段就建立规范的设计文件结构，为后续的 RTL 编码和模块化开发奠定基础。新建的文件会自动添加到当前项目的 Design Sources 分组中，并可在 Sources 面板中查看和管理，如图 5 所示。

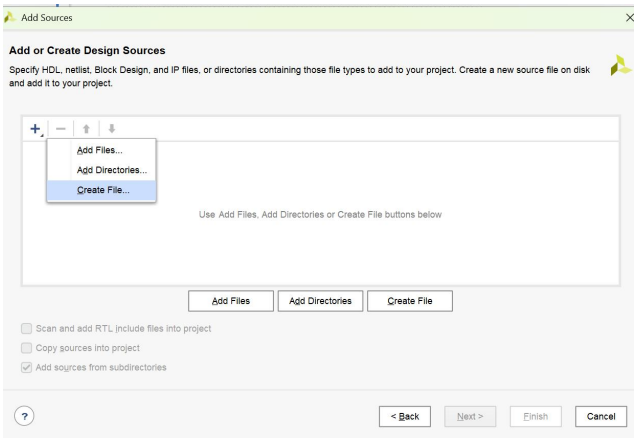


图 5 工程配置阶段

6. 命名文件名称、选择文件位置，点击 OK 完成创建，如图 6 所示。

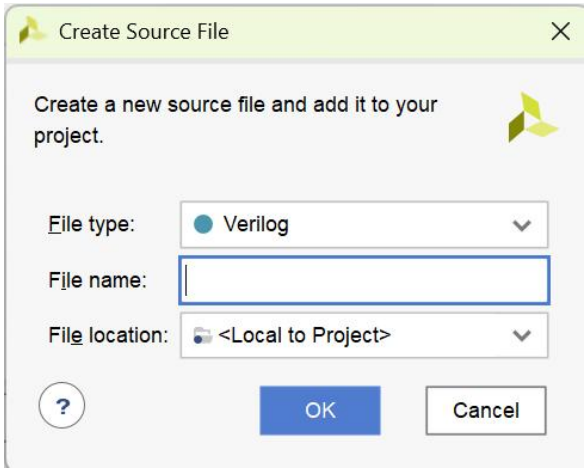


图 6 文件命名

7. 同样的方式创建文件 create simulation sources，最终效果如图 7 所示。

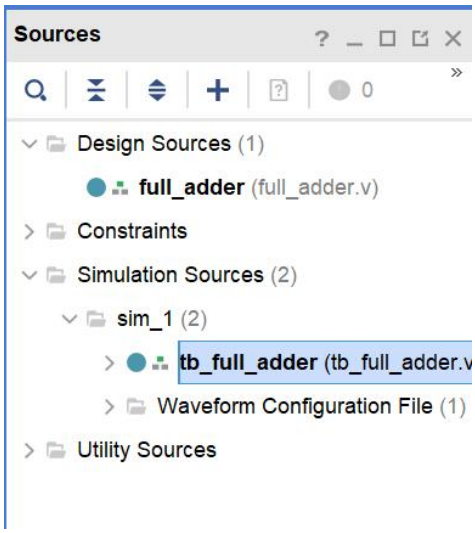


图 7 再创建文件

8. 双击创建的 design sources 进入编辑界面，编写目标程序，如图 8 所示。

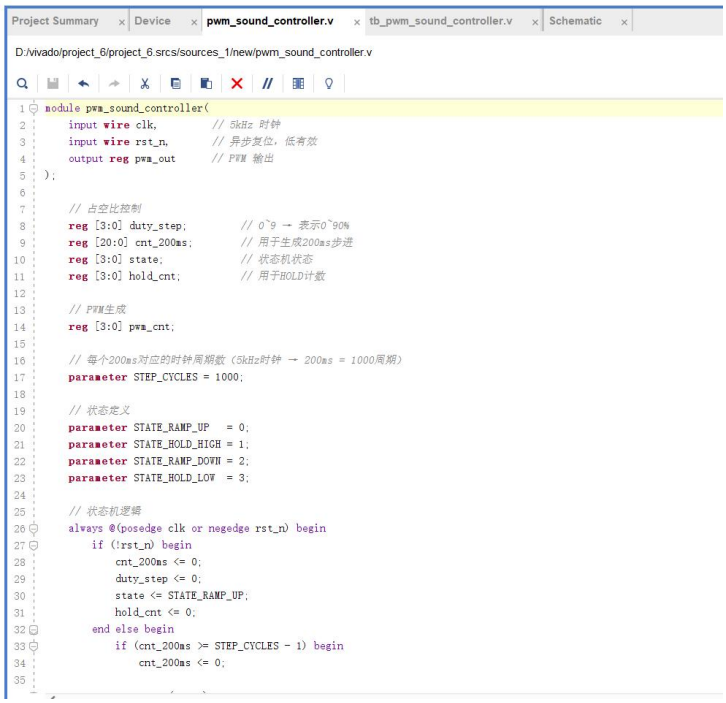


图 8 编写目标程序

具体代码如下所示：

```
module pwm_sound_controller(
    input wire clk,          // 5kHz 时钟
    input wire rst_n,        // 异步复位，低有效
    output reg pwm_out       // PWM 输出
);

    // 占空比控制
    reg [3:0] duty_step;          // 0~9 → 表示 0~90%
    reg [20:0] cnt_200ms;         // 用于生成 200ms 步进
    reg [3:0] state;              // 状态机状态
    reg [3:0] hold_cnt;           // 用于 HOLD 计数

    // PWM 生成
    reg [3:0] pwm_cnt;

    // 每个 200ms 对应的时钟周期数（5kHz 时钟 → 200ms = 1000 周期）
    parameter STEP_CYCLES = 1000;
```

```
        // 状态定义
        parameter STATE_RAMP_UP   = 0;
        parameter STATE_HOLD_HIGH = 1;
        parameter STATE_RAMP_DOWN = 2;
        parameter STATE_HOLD_LOW  = 3;

        // 状态机逻辑
        always @(posedge clk or negedge rst_n) begin
            if (!rst_n) begin
                cnt_200ms <= 0;
                duty_step <= 0;
                state <= STATE_RAMP_UP;
                hold_cnt <= 0;
            end else begin
                if (cnt_200ms >= STEP_CYCLES - 1) begin
                    cnt_200ms <= 0;

                    case (state)
                        STATE_RAMP_UP: begin
                            if (duty_step < 9) begin
                                duty_step <= duty_step + 1;
                            end else begin
                                state <= STATE_HOLD_HIGH;
                                duty_step <= 10;  // 100%
                                hold_cnt <= 0;
                            end
                        end

                        STATE_HOLD_HIGH: begin
                            if (hold_cnt < 4) begin  // 5 × 200ms = 1
秒
                                hold_cnt <= hold_cnt + 1;
                            end else begin
                                hold_cnt <= 0;
                                state <= STATE_RAMP_DOWN;
                                duty_step <= 9;
                            end
                        end

                        STATE_RAMP_DOWN: begin
                            if (duty_step > 0) begin
                                duty_step <= duty_step - 1;
                            end else begin
```

```
                        state <= STATE_HOLD_LOW;
                        hold_cnt <= 0;
                    end
                end

                STATE_HOLD_LOW: begin
                    if (hold_cnt < 4) begin  // 1秒
                        hold_cnt <= hold_cnt + 1;
                    end else begin
                        hold_cnt <= 0;
                        state <= STATE_RAMP_UP;
                        duty_step <= 0;
                    end
                end
            endcase
        end else begin
            cnt_200ms <= cnt_200ms + 1;
        end
    end
end

// PWM 输出逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        pwm_cnt <= 0;
        pwm_out <= 0;
    end else begin
        pwm_cnt <= (pwm_cnt == 9) ? 0 : pwm_cnt + 1;
        pwm_out <= (pwm_cnt < duty_step) ? 1'b1 : 1'b0;
    end
end

Endmodule
```

9.打开 simulation sources，编写测试文件，如图 9 所示。

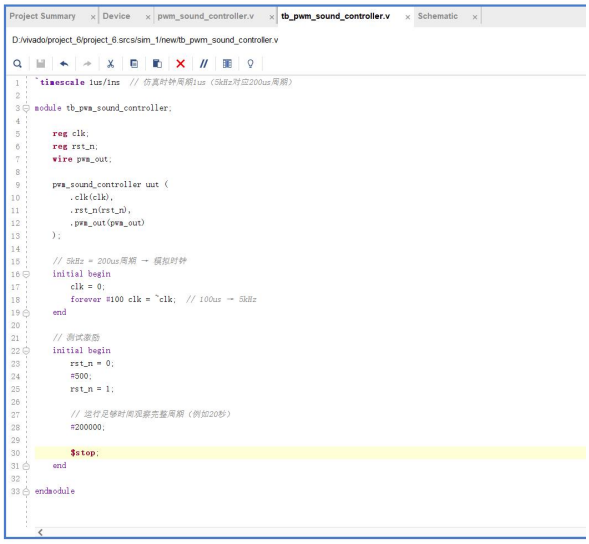


图 9 编写测试文件

具体代码如下所示：

```
`timescale 1us/1ns    //  仿真时钟周期 1us（5kHz 对应 200us 周期）

module tb_pwm_sound_controller;

    reg clk;
    reg rst_n;
    wire pwm_out;

    pwm_sound_controller uut (
        .clk(clk),
        .rst_n(rst_n),
        .pwm_out(pwm_out)
    );

    // 5kHz = 200us 周期  →  模拟时钟
    initial begin
        clk = 0;
        forever #100 clk = ~clk;    // 100us → 5kHz
    end

    //  测试激励
    initial begin
        rst_n = 0;
        #500;
        rst_n = 1;

        //  运行足够时间观察完整周期（例如 20 秒）
        #200000;
```

```
        $stop;
    end

endmodule
```

10.在 Vivado 仿真流程中，首先需在 Sources 面板中右键单击仿真源文件（如 testbench 文件），选择"Set as Top"将其设置为顶层仿真模块，然后展开 Flow Navigator 中的 Simulation 选项，点击"Run Simulation"启动行为级仿真，如图 10 所示。

该操作会以当前设置的顶层测试文件作为仿真入口，自动调用 Vivado 自带的仿真器进行 RTL 级功能验证，并在仿真完成后显示波形窗口和仿真结果报告。

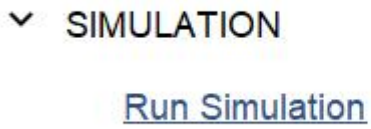


图 10　启动窗口

11.仿真完成后，系统会自动生成波形文件并弹出仿真波形窗口（通常为.wcfg 格式），该窗口将可视化显示所有测试信号在仿真周期内的时序变化情况，如图 11 所示。

用户可通过此波形文件直观验证设计逻辑的正确性。

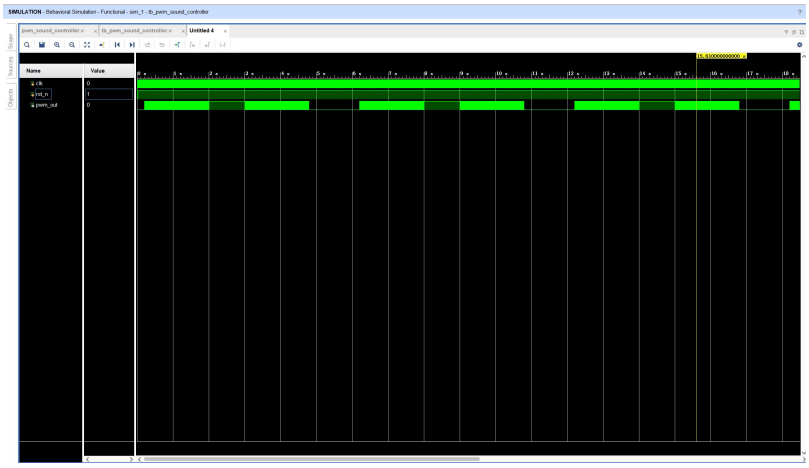


图 11 仿真波形窗口

左上角四个箭头的按钮，可以进行展开，便于观察图形。

可以观察到以下几点信息：

1. 时钟信号（clk）

clk 是一个稳定的时钟信号，频率为 5KHz。这与实验要求中的时钟频率一致。时钟周期为 0.2 毫秒，符合预期。

2. 复位信号（rst_n）

rst_n 是一个高电平有效的复位信号，在整个仿真过程中保持高电平状态，表示系统没有被复位。

3. PWM 输出信号（pwm_out）

pwm_out 是变音器的输出信号，其波形显示了声音强度随时间变化的过程。

波形呈现出周期性的变化，符合实验要求中的“从无声到响亮，再从响亮到无声”的效果。

具体分析：

静音阶段：在某些时间段内，pwm_out 信号完全为低电平，表示蜂鸣器处于静音状态。

音量上升阶段：pwm_out 信号的占空比逐渐增加，从 0%逐步变化到 100%，表示音量逐渐增大。

最强音阶段：在某个时间段内，pwm_out 信号的占空比保持在 100%，表示蜂鸣器达到最响亮的状态。

音量下降阶段：pwm_out 信号的占空比逐渐减小，从 100%逐步变化到 0%，表示音量逐渐减小。

再次静音阶段：pwm_out 信号再次回到完全低电平状态，表示蜂鸣器再次进入静音状态。

从仿真结果图可以看出，设计的变音器能够按照预期实现声音强度随时间变化的效果。具体表现为：

音量从无声逐渐增大到最响亮，然后又逐渐减小到无声，周而复始。

每个阶段的时间分配符合实验要求，即音量提高过程耗时 2 秒，最强音处维持 1 秒，音量降低过程耗时 2 秒，静音处维持 1 秒。

12. 在完成 Verilog 代码编写后，可通过以下步骤生成对应的 RTL 电路图：首先在 Vivado 的"Flow Navigator"面板中点击"Run Synthesis"启动综合流程，如图 12 所示。

该操作会将 HDL 代码转换为门级网表。

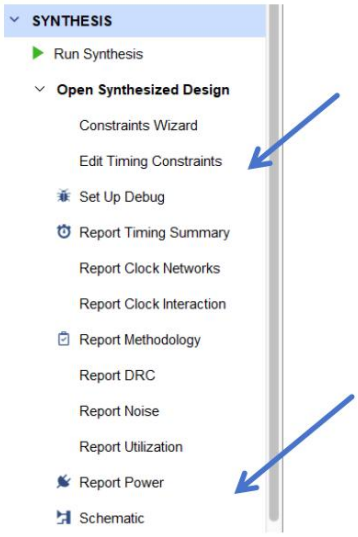


图 12 操作提示

13.综合完成后，在综合结果窗口选择"Open Synthesized Design"，然后通过菜单栏的"Tools"→"Schematic"查看自动生成的电路原理图，如图 13 所示。

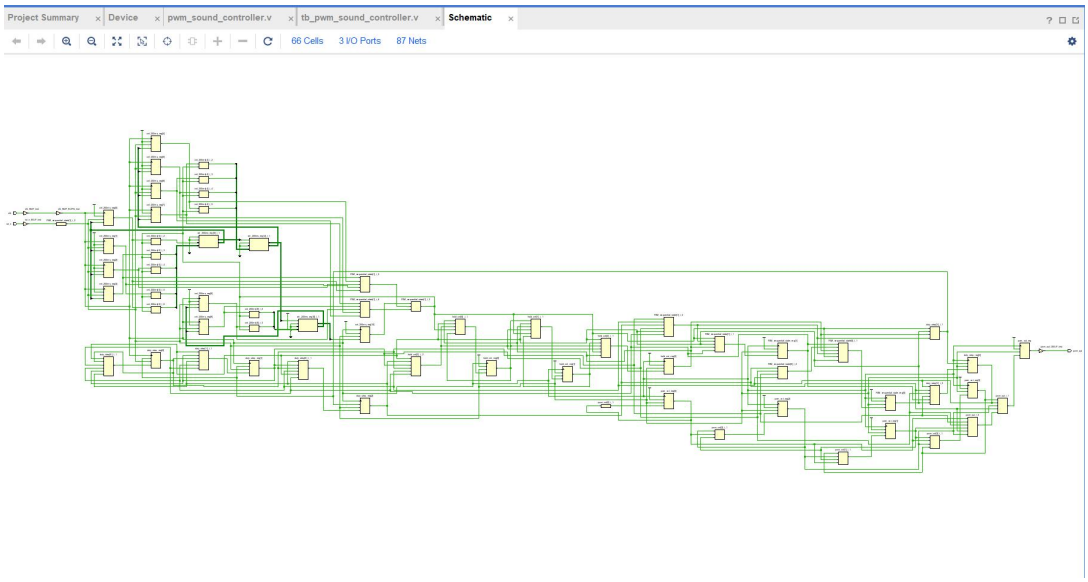


图 13 电路原理图

从提供的电路原理图项目摘要中，可以提取以下关键硬件实现信息：

设计规模与结构特征

（1）逻辑规模：60 个 Cells（包含 LUT/FF 等基本单元），属于中小规模设计

（2）互联复杂度：87 条 Nets 显示模块间信号交互较为精简

硬件实现特点

（1）纯逻辑架构：未见 BRAM/DSP 等专用单元调用

（2）紧凑型设计：低 Cell/Nets 比例（≈1:1.45）表明：

a) 采用高效的状态机实现核心控制

b) 可能复用部分计算逻辑（如共用计数器）

四、实验感悟

通过本次"咏叹调"变音器的 Verilog 设计与实现，我获得了许多宝贵的经验和体会：

（1）理论与实践的结合

PWM 原理和状态机设计在本次实验中得到了实际应用。将抽象的占空比控制转化为具体的音量变化效果，让我深刻理解了数字信号调制在实际硬件中的实现方式。

（2）时序控制的重要性

实验中需要精确控制各个阶段的时间（0.2 秒/档，1 秒保持等），这让我认识到在硬件设计中时序控制的严谨性。通过调试计数器参数，我学会了如何计算和验证不同时间尺度下的时钟周期数。

（3）状态机设计的艺术

设计清晰的四状态（上升、保持、下降、静音）状态机是项目成功的关键。这让我明白复杂时序逻辑可以通过合理的状态划分变得清晰可控，状态转换条件的精确设计直接影响系统行为的正确性。

（4）调试技巧的提升

在实现过程中，我遇到了音量变化不均匀、状态转换不及时等问题。通过分段仿真和信号观察，逐步定位问题原因，这种调试经验对提高我的硬件调试能力非常有帮助。

（5）硬件资源的合理利用

考虑到计数器位宽和比较器的实现，我学会了在满足功能需求的前提下优化设计，平衡资源占用和性能。

（6）创新思维的培养

在完成基础要求后，我尝试了修改参数实现不同的音效变化模式（如非线性变化），这启发我在工程实践中保持创新思维。

这次实验不仅巩固了我的 Verilog 编程能力，更让我体会到数字系统设计的乐趣。当听到蜂鸣器按照设计发出预期的"咏叹"音效时，获得的成就感是单纯理论学习无法比拟的。这激励我在今后的学习中更加注重实践环节，培养解决实际工程问题的能力。